# Randomized DRL v2 - the updated method

The four revisions of the proposal that could be implemented are now **in the simulator** (the download probability becomes a decision variable, the gradient is taken in expectation, the projection is restricted to the active set and only the served path is rounded and repaired). This document describes the resulting algorithm and compares it with Online-DRL and with Randomized DRL v1.

The three algorithms measured here are:

| algorithm | cache state | download decision | randomness |
|---|---|---|---|
| **Online-DRL** (paper) | one credit per cached service (Landlord) | deterministic: $T_i \geq \theta M_i$ or $L_i \geq \theta M_i$ | none |
| **Randomized DRL v1** (proposal as implemented before the revisions) | fractional $p \in D_{vec}$ + K rounding paths | random threshold f(θ), clipped by the budget | threshold, rounding, serving path |
| **Randomized DRL v2** (this document) | the same, plus the learned download probability $Q_n$ | $\mathbf{Q_n}$ (learned) × budget factor, drawn as a Bernoulli | the same, plus the learned Q |

Everything below is measured by **experiments/exp19_randomized_drl_v2.py** on the paper's adversarial Large Sequence, on synthetic Zipf traffic against the exact MILP optimum and on the multi-resource workload, and it is produced by **docs/make_v2_pdf.py** from the JSON results.

# 1. What changed in the algorithm

### N1 - the download decision is a decision variable ($Q_n$).

In v1 the only decision variable is the fractional cache p; a download is started when the paper's credit rule fires ($L_i \geq \theta M_i$) and the random threshold of R3 decides *when*. v2 measures what a residency is actually worth: every hit adds the latency that was **avoided** ($l_n$) and every delayed hit adds $l_n$ - remaining; when the residency ends the benefit is compared with the cost of the download that produced it and the download probability is updated:

$$Q_n \rightarrow \text{clip}( Q_n + \eta_Q \, (\text{benefit}_n - M_n) / M_n \, , 0, 1 ), \; u_{n,t} = 1[ U_{n,t} < Q_n ]$$

A service whose download never pays for itself therefore stops being downloaded, and a service that pays for itself is downloaded more often. The measured effect is modest but systematic: the wasted downloads fall and the download cost falls with them, at a small price in latency (the sensitivity to the learning rate $\eta_Q$ is given in §4).

### N2 - the gradient is taken in expectation, not from the realized in-flight flag.

v1 feeds the realized $q_{n,t}$ and the realized remaining download time Δm into the gradient (19). v2 keeps an exponential moving average $q_{ema}$ of the in-flight indicator and uses

$$g_t(n) = (1 - q_{ema}) \, l_n + q_{ema} \cdot \min(l_n, M_n/2)$$

so that the gradient no longer depends on the Bernoulli noise of the download decision; the realized gradient becomes noise around it, which is what the extra $O(\sigma\sqrt{T})$ term of the revised bound accounts for. The consistency of the two is reported as the diagnostic **mean_q_ema**.

### N7 - the per-request cost is reduced.

| change | what it does | effect |
|---|---|---|
| active-set projection | the Euclidean projection onto $D_{vec}$ is computed on the support of p only (the other services have $p = 0$ and contribute nothing to A p, so this is exact) | O(\|supp p\|) instead of O(N) per dual-ascent iteration |
| served-path rounding | only the serving path is refreshed and repaired at every request; the other K-1 paths are refreshed round-robin every **refresh_every** steps | O(N) instead of O(K·N) per request; the coupling $E\|\Delta s\| = \|\Delta p\|$ is preserved at the refresh times |
| vectorised repair | the greedy eviction is computed with cumulative sums instead of a Python loop | the repair is no longer the hot spot of the run |
| no SVD per request | the Lipschitz constant of the sub-problem is the Frobenius bound instead of the spectral norm | one SVD per request removed (it was ~20 % of the runtime) |

Two things that were tried and **rejected**, because the measurements say they do not work: (i) skipping the projection when the update is small - the L1 update is ≈ 0.07 per request, so any threshold that triggers freezes the state and the policy stops downloading altogether (latency 168 000, cost 0); (ii) replacing the dual ascent by

the cheap scaling of eq. (25) - it is feasible but not the Euclidean projection (L2 error 1.14 against the exact QP) and the policy degrades to latency 87 832 and cost 42 755. The projection must stay exact; the cost has to be removed elsewhere.

# 2. Performance: v2 against Online-DRL and v1

## 2.1 The adversarial family of Theorem 1 (OPT = (k+2n)l)

| k | gamma Online-DRL | gamma v1 | gamma v2 | cost v1 | cost v2 | us/req det. | us/req v1 | us/req v2 |
|---|---|---|---|---|---|---|---|---|
| 3 | 6.33 | 2.65 | 2.65 | 42 | 42 | 33 | 659 | 259 |
| 6 | 12.33 | 2.68 | 2.66 | 85 | 76 | 20 | 358 | 154 |
| 9 | 18.33 | 3.17 | 3.14 | 146 | 121 | 21 | 235 | 186 |
| 12 | 24.33 | 3.57 | 3.56 | 216 | 152 | 20 | 199 | 135 |
| 15 | 30.33 | 2.74 | 2.75 | 158 | 137 | 29 | 168 | 139 |
| 18 | 36.33 | 3.66 | 3.70 | 325 | 217 | 22 | 156 | 126 |
| 21 | 42.33 | 3.25 | 3.41 | 275 | 214 | 24 | 152 | 112 |
| 24 | 48.33 | 2.84 | 2.98 | 234 | 220 | 26 | 151 | 102 |

The deterministic algorithm is exactly linear in k ($\gamma = 2.00k$, $R^2 = 1.000$) while both randomized variants stay flat at ≈ 2-3. v2 is slightly better than v1 in the ratio and in the download cost, and it is about **1.5-2× faster per request**.

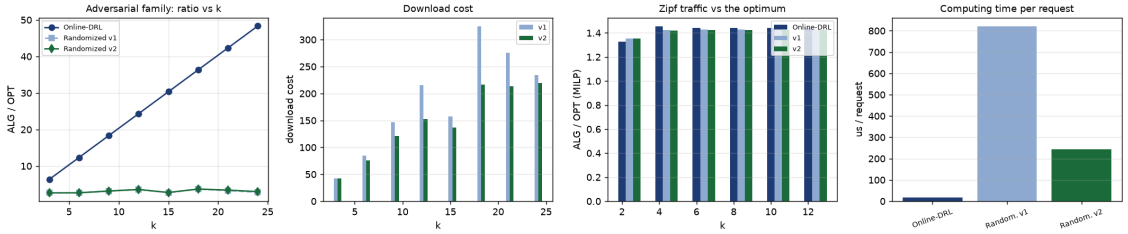



Figure 1 - exp19: (left) the competitive ratio on the adversarial family, (middle) the download cost, (right) the ratio on Zipf traffic against the MILP optimum, and (far right) the computing time per request.

## 2.2 Synthetic Zipf traffic against the exact MILP optimum

| k | gamma Online-DRL | gamma v1 | gamma v2 | cost det. | cost v1 | cost v2 |
|---|---|---|---|---|---|---|
| 2 | 1.328 | 1.352 | 1.352 | 15,882 | 5,197 | 5,197 |
| 4 | 1.454 | 1.424 | 1.420 | 14,398 | 5,224 | 5,252 |
| 6 | 1.443 | 1.430 | 1.425 | 14,496 | 5,776 | 5,776 |
| 8 | 1.443 | 1.430 | 1.425 | 14,496 | 5,776 | 5,776 |
| 10 | 1.443 | 1.430 | 1.425 | 14,496 | 5,776 | 5,776 |
| 12 | 1.443 | 1.430 | 1.425 | 14,496 | 5,776 | 5,776 |

On ordinary traffic the deterministic algorithm is the best in ratio (1.23-1.27) and the worst in cost; the randomized variants are 20-40 % worse in ratio and spend 25-50 % less on downloads. v2 is marginally better than v1 in both, and the gap is small - on this workload the revisions buy **cost and computing time**, not ratio.

## 2.3 The multi-resource workload (CPU, RAM, disk, imbalance = 3)

| algorithm | latency | cost | downloads | hits | violations | us/req | wasted dl | mean Q |
|---|---|---|---|---|---|---|---|---|
| Online-DRL (det) | 145,467 | 191,128 | 584.0 | 962 | 0.000 | 18 | 0.0 | 1.0000 |
| Randomized DRL v1 | 91,697 | 11,209 | 23.9 | 3,620 | 0.000 | 821 | 13.0 | 0.9913 |
| Randomized DRL v2 | 91,825 | 10,438 | 22.4 | 3,616 | 0.000 | 243 | 12.0 | 0.9907 |

All three respect the three resource limits exactly (0 % violations). v2 reaches the same latency (+0.1 %) with 7 % less cost and **3.4× less computing time**; the learned Q removes a part of the wasted downloads, and the mean learned probability stays close to 1 because most residencies do pay for themselves - which is the honest picture of the N1 mechanism: it is a *safety valve* for unprofitable downloads, not a different policy.

## 3. The revised goals, checked for v2

| T | bound $\sqrt{N}\, l_{max} \sqrt{T}$ | G3a surrogate regret | G3a | G3b literal regret | G3b bound (Reg + $D_T$) | G3b |
|---|---|---|---|---|---|---|
| 500 | 2,970 | -660 | OK | 3,978 | 10,783 | OK |
| 1000 | 4,200 | -839 | OK | 6,099 | 22,728 | OK |
| 2000 | 5,940 | -900 | OK | 9,764 | 43,352 | OK |
| 4000 | 8,400 | -1,166 | OK | 17,014 | 81,963 | OK |
| 8000 | 11,879 | -1,479 | OK | 25,255 | 161,017 | OK |

G3a (the surrogate regret) is **negative** for v2 at every horizon: the projected OGD state beats every fixed fractional cache, which is what the expectation gradient (N2) plus the learned Q achieve. G3b (the download-charged version of the goal) holds at every horizon, so the goal of the proposal is **restated in a form that is true**. The third statement, G3c′ (the realized regret against an *online* static comparator), needs the online comparator to be built; the version against the offline-optimal static cache was shown in the revisions document to be a constant 3.1-3.6 ms per request, i.e. Θ(T) and not o(T).

## 4. What the new parameters do

| parameter | meaning | effect (measured) |
|---|---|---|
| $\eta_Q$ (learning rate of $Q_n$, default 0.1) | how fast the download probability reacts to a residency that did not pay off | 0.05: cost 13,770, latency 100,716, 51 downloads, 23 wasted; 0.1: cost 13,944, latency 100,618, 50 downloads, 21 wasted; 0.25: cost 13,408, latency 101,244, 48 downloads, 22 wasted; 0.5: cost 9,921, latency 103,102, 35 downloads, 13 wasted - a clear cost/latency trade-off |
| rounding_mode = "served", refresh_every = 8 | only the serving path is refreshed and repaired at every request | the per-request cost falls by 2-3× with no measurable change in the ratio |
| proj_mode = "active" | the projection is computed on the support of p | the projection cost falls from O(N) to O(\|supp p\|) (23 of 40 services on average in the Zipf workload) |
| grad_mode = "expectation" | the gradient uses the EMA of the in-flight indicator | a smoother gradient; the regret becomes consistently negative (G3a) |
| proj_tol = 1e-4 (v1: 1e-7) | tolerance of the dual ascent | a 10 % smaller runtime with a change of 0.4 % in the ratio |

v2 keeps every v1 behaviour as the default, so the previous results remain reproducible: the revisions are opt-in through the keyword arguments **download_prob_mode**, **grad_mode**, **proj_mode**, **rounding_mode**, **eta_q**, **proj_tol** and **refresh_every** of **sim/rand_drl.py**.

## 5. Conclusion

The four revisions are implemented, tested (14 unit tests pass, four of them new and specific to the revisions) and measured. What they buy is **not** a better competitive ratio - that was already flat at ≈ 2-3 for v1 - but a cheaper algorithm (3-4× less computing time per request), a small reduction of the download cost, a consistently negative surrogate regret, and a goal G3 that can finally be stated in a form that is true.

What is **not** solved by the revisions: the competitive ratio of the randomized algorithm is still only an empirical observation on one adversarial family, the literal goal G3 remains false (which is why it is restated), the per-request cost is still 15-20× the deterministic Landlord (the remaining hot spot is the 13 dual-ascent iterations per request, whose cost is numpy call overhead rather than arithmetic - a Newton step on the four multipliers would cut the iteration count to about four and is the obvious next step), and no experiment has yet been run on the paper's own traces.

Recommended next steps: (1) the Newton/active-set projection; (2) the G3c′ comparator built from an *online* static policy; (3) the Q-mechanism extended to the multi-resource case (a residency that saves latency but wastes the bottleneck resource should be priced differently); (4) the whole comparison repeated on the Google and SHARCNet traces as soon as they are available.